

Lab 1 — Echo Server

Complete Explanation & Discussion Guide

CC451 — Computer Networks

Alexandria University • Spring 2026

Component	File	Purpose
TCP Server	tcp_server.py	Multi-threaded TCP echo server
UDP Server	udp_server.py	Connectionless UDP echo server
Client	client.py	Unified client for both TCP & UDP

Table of Contents

- 1. High-Level Overview**
 - 1.1 Architecture Diagram
 - 1.2 How the Files Work Together
- 2. Message Processing Logic**
- 3. TCP Server — Full Code Walkthrough**
 - 3.1 Imports & Constants
 - 3.2 `process_message()` Function
 - 3.3 `handle_client()` — The Thread Worker
 - 3.4 `main()` — Accept Loop & Threading
- 4. UDP Server — Full Code Walkthrough**
 - 4.1 Key Differences from TCP
 - 4.2 The Main Loop
- 5. Client Program — Full Code Walkthrough**
 - 5.1 TCP Client Function
 - 5.2 UDP Client Function
- 6. Key Concepts to Understand**
 - 6.1 TCP vs UDP
 - 6.2 Multi-threading
 - 6.3 Socket API
- 7. Running & Testing the Code**
- 8. Wireshark Guide**
- 9. Possible TA Questions & Answers**

1. High-Level Overview

This lab requires implementing an **Echo Server** system using Python's `socket` module. The system consists of **three files**, each with a distinct role:

File	Role	Protocol	Key Feature
<code>tcp_server.py</code>	Server	TCP	Multi-threaded (one thread per client)
<code>udp_server.py</code>	Server	UDP	Connectionless, single-threaded loop
<code>client.py</code>	Client	TCP & UDP	Unified client, mode chosen via CLI arg

1.1 Architecture Diagram

The communication flow works as follows:

	TCP Flow		UDP Flow	
Step 1	Client connects (3-way handshake)	→	Client sends datagram (no handshake)	→
Step 2	Client sends message over persistent connection	→	Server receives datagram with client address	→
Step 3	Server reads from client's thread		Server processes message	
Step 4	Server sends response back on same connection	←	Server sends response back to client address	←
Step 5	Connection stays open for more messages		No connection to maintain	

1.2 How the Files Work Together

Step 1: You start one of the servers (TCP or UDP) in a terminal. It binds to a port and waits.

Step 2: You start the client in another terminal, specifying `tcp` or `udp` mode. The client connects (TCP) or just prepares a socket (UDP).

Step 3: You type a message like `Ahe l l o`. The client sends it to the server.

Step 4: The server reads the first character, applies the rule, and sends the processed result back.

Step 5: The client prints the server's response. You can keep sending messages or type `quit` to exit.

2. Message Processing Logic

Both the TCP and UDP servers share the **exact same** processing logic. The first character of the message is a **command**, and the rest is the **data** to process.

1st Char	Action	Example Input	Output
'A'	Sort remaining chars DESCENDING (Z→A)	Ahello	ollhe
'C'	Sort remaining chars ASCENDING (A→Z)	Chello	ehllo
'D'	Convert remaining to UPPERCASE	Dhello	HELLO
Other	Echo entire message unchanged	Xhello	Xhello

Key Detail: The command character ('A', 'C', 'D') is **removed** from the output. Only the processed remaining string is returned. However, for the default case (any other first character), the **entire original message** including the first character is echoed back.

Here is the shared `process_message()` function used by both servers:

```
def process_message(message: str) -> str:
    if len(message) == 0:          # guard against empty input
        return message

    command = message[0]           # first character = command
    data    = message[1:]          # rest = data to process

    if command == 'A':             # A → sort DESCENDING
        return ''.join(sorted(data, reverse=True))
    elif command == 'C':           # C → sort ASCENDING
        return ''.join(sorted(data))
    elif command == 'D':           # D → UPPERCASE
        return data.upper()
    else:                          # anything else → echo as-is
        return message
```

How sorted() works: Python's `sorted()` sorts characters by their ASCII/Unicode value. Lowercase letters a-z have values 97-122, so `sorted('hello')` gives `['e', 'h', 'l', 'l', 'o']`. The `reverse=True` parameter flips this to descending order.

3. TCP Server — Full Code Walkthrough

The TCP server is the most complex file because it must handle **multiple simultaneous clients** using **multi-threading**. Each client gets its own dedicated thread.

3.1 Imports & Constants

```
import socket      # provides the socket API for network communication
import threading   # allows creating threads for concurrent clients

HOST = '127.0.0.1' # localhost — only accepts connections from this machine
PORT = 65432       # the port number the server listens on
```

127.0.0.1 is the **loopback address** — it means the server only accepts connections from the same computer. Port 65432 is in the ephemeral/dynamic range (49152-65535), avoiding conflicts with well-known services.

3.2 process_message() Function

Already explained in Section 2. This function is **identical** in both servers. It takes a raw string, inspects the first character, and returns the processed result.

3.3 handle_client() — The Thread Worker

This is the most important function to understand for the TA discussion. It runs in a **separate thread** for each connected client.

```
def handle_client(conn: socket.socket, addr: tuple):
    print(f"[TCP] New connection from {addr}")
    with conn:
        # 'with' ensures socket closes when done
        while True:
            # keep reading messages from this client
            try:
                data = conn.recv(1024) # read up to 1024 bytes
                if not data:           # empty data = client disconnected
                    break
                message = data.decode('utf-8').strip()
                response = process_message(message)
                conn.sendall(response.encode('utf-8'))
            except ConnectionResetError:
                break # client crashed / force-closed
            except Exception as e:
                print(f"[TCP] Error with {addr}: {e}")
                break
    print(f"[TCP] Connection closed: {addr}")
```

Block-by-block breakdown:

Line / Block	What It Does	Why It Matters
with conn:	Context manager — auto-closes socket when block exits	Prevents resource leaks even if an exception occurs
while True:	Infinite loop — keeps reading messages from this client	TCP is persistent; one connection can carry many messages
conn.recv(1024)	Reads up to 1024 bytes from the client	Blocks (waits) until data arrives or client disconnects

<code>if not data: break</code>	Checks if client sent empty bytes (= disconnect)	Graceful way to detect client disconnection
<code>data.decode('utf-8')</code>	Converts raw bytes to a Python string	Network sends bytes, not strings; we must decode them
<code>.strip()</code>	Removes leading/trailing whitespace and newlines	Clients may send <code>\n</code> at the end of their messages
<code>conn.sendall()</code>	Sends the full response back to the client	Unlike <code>send()</code> , <code>sendall()</code> guarantees ALL bytes are transmitted
<code>ConnectionResetError</code>	Caught when client crashes or force-closes connection	Prevents server from crashing when a client dies unexpectedly

3.4 main() — Accept Loop & Threading

```
def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print(f"[TCP] Server listening on {HOST}:{PORT}")

    try:
        while True:
            conn, addr = server.accept() # blocks until a client connects
            thread = threading.Thread(
                target=handle_client,
                args=(conn, addr),
                daemon=True
            )
            thread.start()
    except KeyboardInterrupt:
        print("\n[TCP] Server shutting down.")
    finally:
        server.close()
```

Block-by-block breakdown:

Line / Block	Explanation
<code>socket.AF_INET</code>	Use IPv4 addressing (as opposed to <code>AF_INET6</code> for IPv6)
<code>socket.SOCK_STREAM</code>	Use TCP protocol (stream-oriented, reliable, ordered)
<code>SO_REUSEADDR</code>	Allows restarting the server immediately after stopping it, without waiting for the OS to release the port (<code>TIME_WAIT</code> state)
<code>server.bind((HOST, PORT))</code>	Associates the socket with a specific IP and port number
<code>server.listen()</code>	Puts the socket in listening mode, ready to accept connections. The OS maintains a backlog queue of pending connections.
<code>server.accept()</code>	BLOCKS until a new client connects. Returns a NEW socket (<code>conn</code>) dedicated to that client, plus the client's address.
<code>threading.Thread(...)</code>	Creates a new thread that will run <code>handle_client()</code> . This is how we handle multiple clients at the same time.
<code>daemon=True</code>	Daemon threads are killed automatically when the main program exits. Without this, the program might hang on exit.
<code>thread.start()</code>	Actually starts the thread running. After this, the main loop immediately goes back to <code>accept()</code> to wait for the next client.
<code>KeyboardInterrupt</code>	Catches Ctrl+C so the server shuts down gracefully

The Threading Flow (Critical to understand):

1. Main thread calls `accept()` and BLOCKS waiting.
2. Client A connects → `accept()` returns → main thread spawns Thread-A.
3. Main thread loops back to `accept()` and BLOCKS again.
4. Client B connects → spawns Thread-B. Meanwhile Thread-A is handling Client A.
5. Both threads run concurrently, each with their own `conn` socket.

4. UDP Server — Full Code Walkthrough

4.1 Key Differences from TCP

Aspect	TCP Server	UDP Server
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection	listen() + accept()	No connection needed
Receiving	conn.recv() on per-client socket	recvfrom() returns data + address
Sending	conn.sendall() on per-client socket	sendto() specifying client address
Threading	Required (one per client)	Not needed (single loop handles all)
Reliability	Guaranteed delivery, ordered	No guarantee, may arrive out of order
State	Maintains connection state	Stateless — each message independent

4.2 The Main Loop

```
def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # DGRAM = UDP
    server.bind((HOST, PORT))
    print(f"[UDP] Server listening on {HOST}:{PORT}")

    try:
        while True:
            data, client_addr = server.recvfrom(1024)  # get data + sender address
            message = data.decode('utf-8').strip()
            response = process_message(message)
            server.sendto(response.encode('utf-8'), client_addr)  # reply to sender
    except KeyboardInterrupt:
        print("\n[UDP] Server shutting down.")
    finally:
        server.close()
```

Line / Block	Explanation
SOCK_DGRAM	Creates a UDP socket (datagram-oriented) instead of TCP (stream)
No listen() or accept()	UDP is connectionless — no handshake. Just bind and start receiving.
recvfrom(1024)	Returns BOTH the data AND the sender's address (IP, port). This is crucial because UDP has no persistent connection.
sendto(data, addr)	Sends response to a specific address. We must explicitly specify WHERE to send because there's no connection to track.
No threading needed	Each recvfrom() gets one complete datagram. Process it, reply, then loop back. The single loop handles all clients sequentially.

Common Misconception: The UDP server does NOT need threading. Because it's connectionless and each datagram is a complete, independent message, the server can handle requests one at a time in a simple loop. There's no persistent connection to maintain.

5. Client Program — Full Code Walkthrough

The client program is a single file that supports both TCP and UDP modes. You choose the mode by passing `tcp` or `udp` as a command-line argument.

5.1 TCP Client Function

```
def tcp_client():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((TCP_HOST, TCP_PORT))    # initiate 3-way handshake
        while True:
            message = input("You: ")
            if message.lower() == 'quit':
                break
            sock.sendall(message.encode('utf-8'))    # send over persistent connection
            response = sock.recv(1024).decode('utf-8')
            print(f"Server: {response}")
```

Key points: The TCP client calls `connect()` once, which triggers the **TCP 3-way handshake** (SYN → SYN-ACK → ACK). After that, it reuses the same connection for all messages. The `with` statement ensures the socket is closed when the client exits, which triggers the TCP connection teardown (FIN → ACK).

5.2 UDP Client Function

```
def udp_client():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        while True:
            message = input("You: ")
            if message.lower() == 'quit':
                break
            sock.sendto(message.encode('utf-8'), (UDP_HOST, UDP_PORT))    # specify dest each time
            response, _ = sock.recvfrom(1024)
            print(f"Server: {response.decode('utf-8')}")
```

Key points: No `connect()` call. Each message is sent independently with `sendto()`, specifying the destination address every time. The `recvfrom()` call returns both the data and the sender's address (we discard the address with `_` since we know it's from the server).

TCP vs UDP Client — Summary:

TCP: `connect()` once → `send()/recv()` in a loop → connection auto-closes

UDP: no `connect` → `sendto()/recvfrom()` in a loop → no connection to close

6. Key Concepts to Understand

6.1 TCP vs UDP

Property	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless (no handshake)
Reliability	Guaranteed delivery (ACKs + retransmission)	Best-effort delivery (packets may be lost)
Ordering	Data arrives in order (sequence numbers)	No ordering guarantee
Speed	Slower due to overhead (handshake, ACKs, flow control)	Faster, lower overhead (no connection setup)
Data boundary	Byte stream — no message boundaries	Datagram — each send is one complete message
Use cases	HTTP, FTP, SSH, email	DNS, streaming, gaming, VoIP
Socket type	SOCK_STREAM	SOCK_DGRAM

6.2 Multi-threading

Why do we need threading for TCP but not UDP?

TCP connections are **persistent**. When a client connects, the server must maintain a dedicated communication channel with that client. If the server's main loop is busy reading from Client A (blocked on `recv()`), it cannot accept Client B's connection. Threading solves this by running each client handler in parallel.

UDP is **stateless**. Each message (datagram) is self-contained and includes the sender's address. The server processes one message, replies, and immediately moves to the next. There's no persistent connection blocking the loop, so a single thread suffices.

6.3 Socket API — Key Functions

Function	Protocol	Description
<code>socket()</code>	Both	Creates a new socket object
<code>bind()</code>	Both	Associates socket with IP + port
<code>listen()</code>	TCP only	Puts socket in server/listening mode
<code>accept()</code>	TCP only	Waits for and accepts a client connection
<code>connect()</code>	TCP only	Client initiates connection to server
<code>send()</code> / <code>sendall()</code>	TCP	Sends data over established connection
<code>recv()</code>	TCP	Receives data from established connection

<code>sendto()</code>	UDP	Sends datagram to specified address
<code>recvfrom()</code>	UDP	Receives datagram + sender address
<code>close()</code>	Both	Closes the socket and releases resources

7. Running & Testing the Code

TCP Test

```
# Terminal 1 – Start the TCP server:
$ python tcp_server.py
[TCP] Server listening on 127.0.0.1:65432

# Terminal 2 – Start the TCP client:
$ python client.py tcp
[TCP Client] Connecting to 127.0.0.1:65432...
[TCP Client] Connected! Type 'quit' to exit.

You: Ahello
Server: ollhe

You: Chello
Server: ehlllo

You: Dhello
Server: HELLO

You: Xhello
Server: Xhello

You: quit
[TCP Client] Disconnected.
```

UDP Test

```
# Terminal 1 – Start the UDP server:
$ python udp_server.py
[UDP] Server listening on 127.0.0.1:65433

# Terminal 2 – Start the UDP client:
$ python client.py udp
[UDP Client] Sending to 127.0.0.1:65433
[UDP Client] Ready! Type 'quit' to exit.

You: Apython
Server: ytponh

You: Cpython
Server: hnopty

You: Dpython
Server: PYTHON
```

Testing Multiple TCP Clients

To verify multi-threading works, open **three terminals**: one for the server and two for clients. Send messages from both clients and confirm they both receive responses independently.

8. Wireshark Guide

You need to capture network packets using Wireshark to show the communication between client and server. Here's how to do it:

Step 1: Start Wireshark

Open Wireshark and select the **Loopback** interface (called `lo` on Linux or Loopback: `lo0` on macOS, or Npcap Loopback Adapter on Windows). This captures traffic sent to `127.0.0.1`.

Step 2: Apply Display Filters

```
For TCP traffic:  tcp.port == 65432
For UDP traffic:  udp.port == 65433
```

Step 3: Run Server & Client

With Wireshark capturing, run the server and client. Send a few messages. You should see packets appearing in real-time.

Step 4: What to Look For

Protocol	What You'll See in Wireshark
TCP	1. SYN, SYN-ACK, ACK (3-way handshake) 2. PSH-ACK packets with your message data 3. ACK packets confirming receipt 4. FIN-ACK packets when connection closes
UDP	1. Datagrams from client to server (source port → 65433) 2. Datagrams from server to client (65433 → source port) 3. No handshake, no ACKs, no connection teardown

Tip: To see the actual message content in Wireshark, click on a packet, then look at the bottom pane. Expand the "Data" section to see the raw bytes and the ASCII text of your message.

9. Possible TA Questions & Answers

Below are likely questions a TA might ask during the lab discussion, grouped by topic.

Q1: What is a socket?

A socket is an endpoint for sending and receiving data across a network. It's identified by an IP address and a port number. In Python, the `socket` module provides the API to create and use sockets.

Q2: What's the difference between `AF_INET` and `SOCK_STREAM`?

`AF_INET` specifies the address family (IPv4). `SOCK_STREAM` specifies the socket type (TCP, reliable byte stream). They are different parameters: `AF_INET` tells the socket *what kind of addresses* to use, while `SOCK_STREAM` tells it *how to transmit data*.

Q3: What is the 3-way handshake?

It's how TCP establishes a connection: (1) Client sends SYN, (2) Server replies with SYN-ACK, (3) Client sends ACK. After this, both sides can send data. This happens inside `connect()` on the client and `accept()` on the server.

Q4: Why use port 65432 and not 80 or 8080?

Ports 0-1023 are well-known (reserved for system services like HTTP on 80). Ports 1024-49151 are registered. Ports 49152-65535 are dynamic/private. We use 65432 to avoid conflicts with existing services. Any unreserved port would work.

Q5: Why does the TCP server need multi-threading?

Because `recv()` is a **blocking** call — it pauses execution until data arrives. If the server had only one thread, it would be stuck waiting on one client and unable to accept or serve others. Threading lets each client run independently.

Q6: What happens if a client disconnects unexpectedly?

The `recv()` call returns empty bytes (`b''`) or raises a `ConnectionResetError`. Our code catches both cases and breaks out of the loop, closing that client's socket. Other client threads are unaffected.

Q7: What does `daemon=True` mean in `threading.Thread()`?

A daemon thread runs in the background and is automatically terminated when the main thread exits. Without this, if you press Ctrl+C, the main thread would exit but client threads would keep running, preventing the program from fully shutting down.

Q8: What is the difference between `send()` and `sendall()`?

`send()` may not send all the data at once — it returns the number of bytes actually sent, and you'd need to loop to send the rest. `sendall()` handles this automatically, blocking until ALL bytes are transmitted. Always prefer `sendall()` for TCP.

Q9: Why doesn't the UDP server need threading?

UDP is connectionless. Each `recvfrom()` receives a complete, independent datagram along with the sender's address. The server processes it and immediately moves to the next. There's no persistent connection that blocks the loop.

Q10: What happens if a UDP packet is lost?

Unlike TCP, UDP provides no retransmission. If a packet is lost, neither the client nor the server will know. The client would hang on `recvfrom()` waiting forever. In production, you'd add a timeout with `sock.settimeout()`.

Q11: Why use `recvfrom()` in UDP but `recv()` in TCP?

`recvfrom()` returns both the data AND the sender's address. In UDP, there's no connection, so we need the address to know where to reply. In TCP, the connection socket (`conn`) already knows who it's talking to.

Q12: How does `sorted()` work on strings?

`sorted('hello')` returns `['e', 'h', 'l', 'l', 'o']` because Python sorts characters by their Unicode code points. We use `''.join()` to turn the list back into a string. Adding `reverse=True` gives descending order.

Q13: What does `SO_REUSEADDR` do?

When a server closes, the OS keeps the port in `TIME_WAIT` state for ~60 seconds. Without `SO_REUSEADDR`, you'd get 'Address already in use' error if you restart the server quickly. This option tells the OS to allow reuse immediately.

Q14: What's the difference between TCP and UDP packets in Wireshark?

TCP packets show connection setup (SYN/ACK flags), sequence numbers, acknowledgments, and connection teardown (FIN). UDP packets are much simpler — just source port, dest port, length, and data. No flags, no sequence numbers, no ACKs.

Q15: Why do we capture on the loopback interface?

Because both client and server run on `127.0.0.1` (localhost), the traffic never leaves the machine. It only passes through the loopback interface, so that's where Wireshark must capture.

Q16: What if the message is just a single character like 'A'?

The command is 'A' and the data is an empty string. `sorted('')` returns an empty list, and `''.join([])` returns an empty string. So the server sends back an empty string — no crash.

Q17: Could you use `select()` or `asyncio` instead of threading?

Yes. `select()` uses I/O multiplexing to handle multiple sockets in a single thread. `asyncio` uses an event loop with coroutines. Both are valid alternatives. Threading was specified in the lab requirements.

Q18: What is encode() / decode() and why is it needed?

Network sockets transmit **bytes**, not strings. `encode('utf-8')` converts a Python string to bytes. `decode('utf-8')` converts bytes back to a string. UTF-8 is the standard encoding that supports all characters.

Good luck with your discussion!